SOMAIYA
VIDYAVIHAR

K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Name and Code: Data Structures Lab **(**ITL302)
Semester: III (SYIT A)
Academic Year: 2026-27 (Odd Semester)
Name: Rashi Dawda
Roll no: 23

Experiment No. 09

Aim: To implement the Binary Search technique to efficiently search for a particular element in a sorted dataset using a real-world application.

Objectives:

- To understand the working of the Binary Search algorithm.
- To search for an element in a sorted array.
- To reduce the number of comparisons while searching.
- To understand the time and space complexity of Binary Search.
- To apply Binary Search in real-world applications like searching student roll numbers.

Theory: Binary Search is an efficient searching technique used to find an element in a sorted array. It works by repeatedly dividing the search range into two halves.

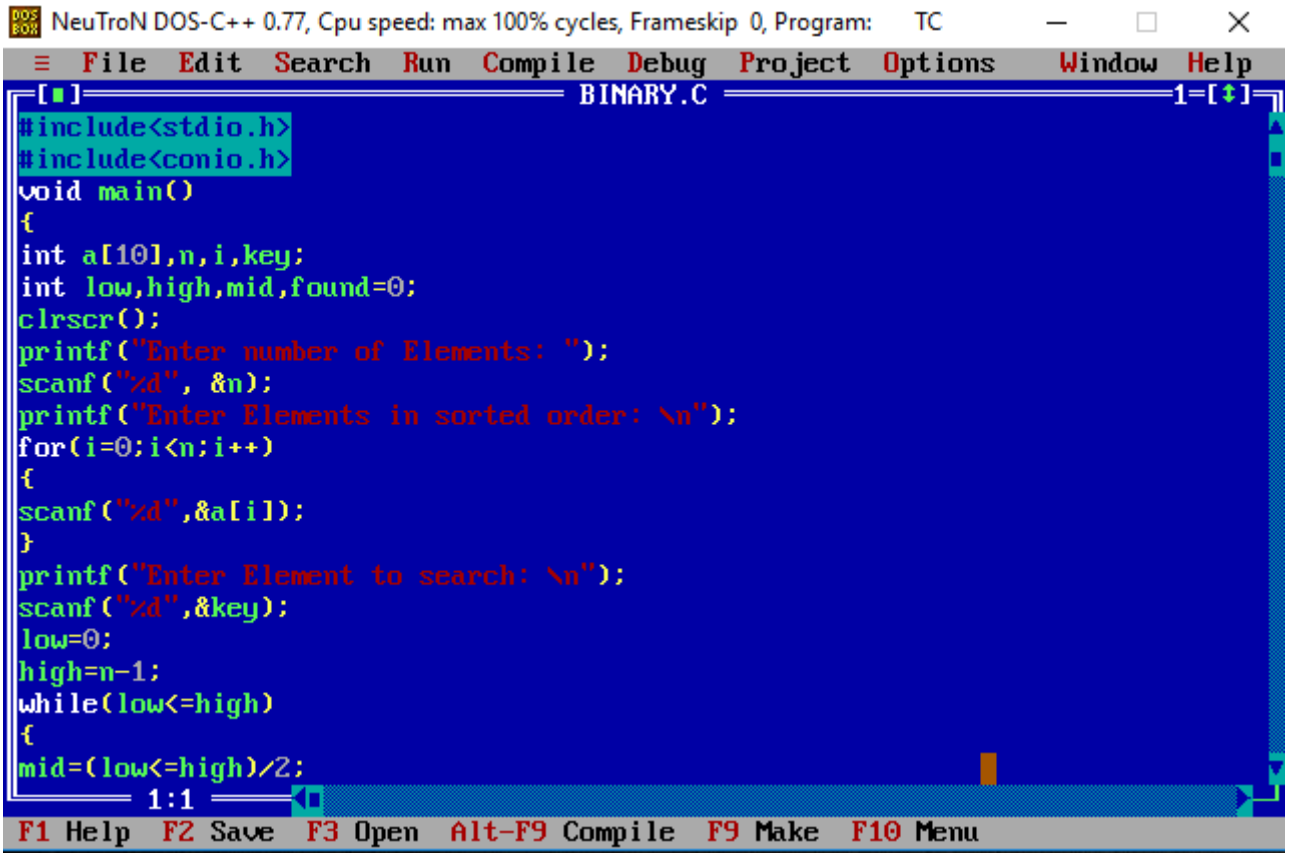
- First, find the middle element of the array.
- If the middle element matches the search value, the search is successful.
- If the search value is smaller, search the left half.
- If the search value is larger, search the right half.
- Repeat until the element is found or the search range becomes empty.

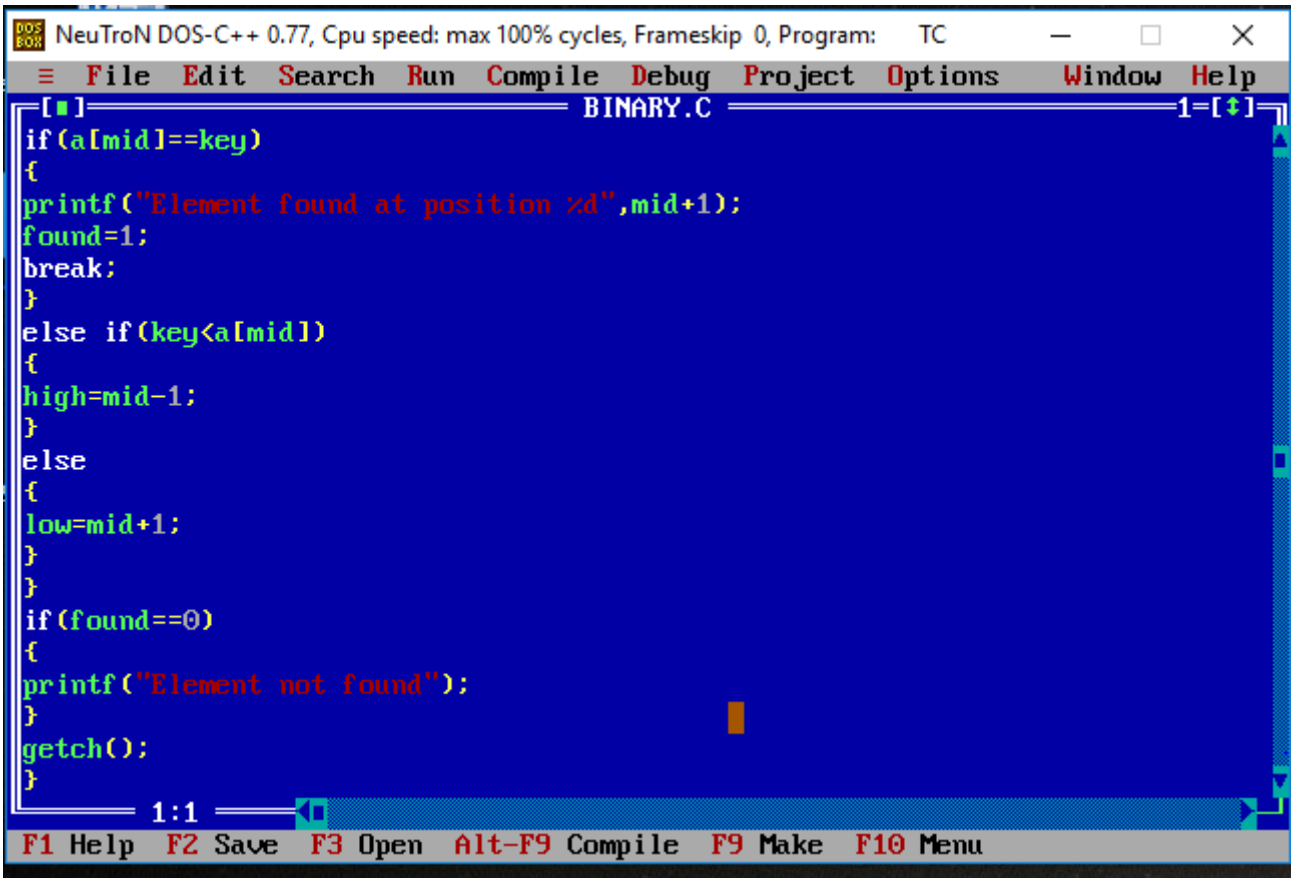
Time Complexity:

- Best Case: O(1)
- Average Case: O(log n)
- Worst Case: O(log n)

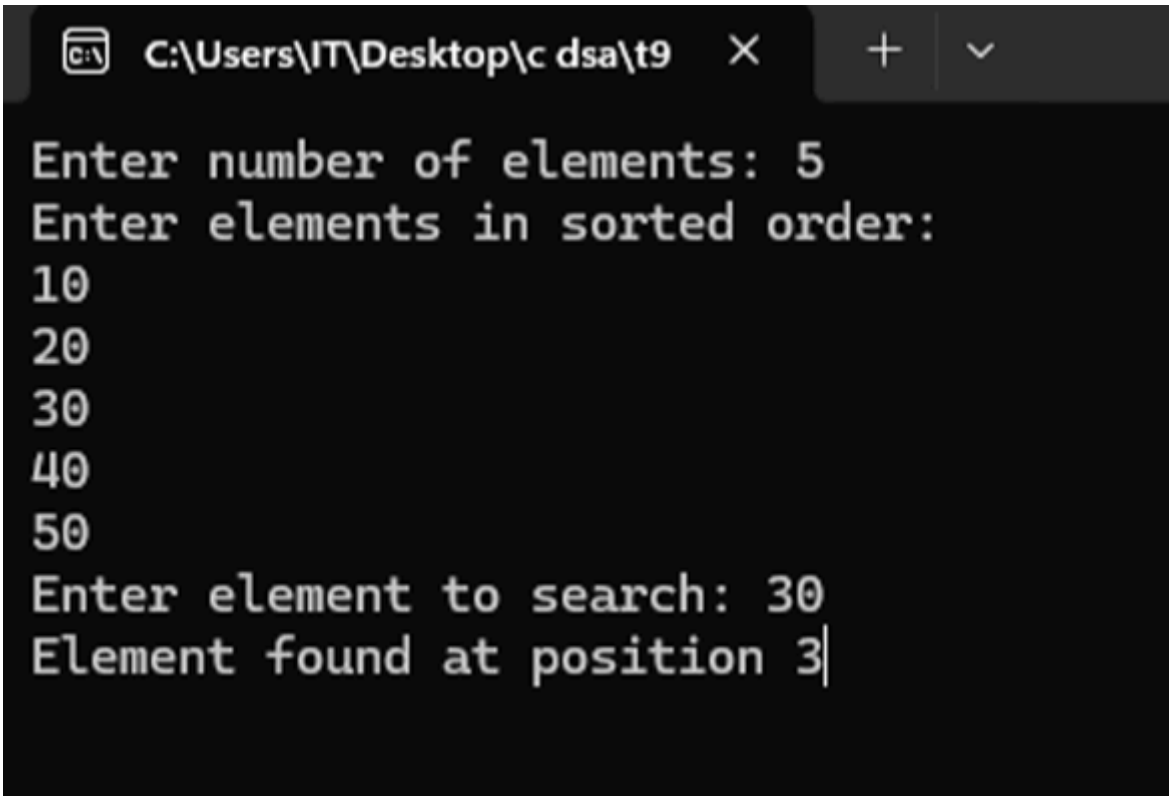
Space Complexity: O(1) for iterative Binary Search.

Code:


NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
File Edit Search Run Compile Debug Project Options Window Help
BINARY.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10],n,i,key;
int low,high,mid,found=0;
clrscr();
printf("Enter number of Elements: ");
scanf("%d", &n);
printf("Enter Elements in sorted order: \n");
for(i=0;i<n;i++)
{
scanf("%d",&a[i]);
}
printf("Enter Element to search: \n");
scanf("%d",&key);
low=0;
high=n-1;
while(low<=high)
{
mid=(low<=high)/2;
1:1
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu


NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
File Edit Search Run Compile Debug Project Options Window Help
BINARY.C
if(a[mid]==key)
{
printf("Element found at position %d",mid+1);
found=1;
break;
}
else if(key<a[mid])
{
high=mid-1;
}
else
{
low=mid+1;
}
}
if(found==0)
{
printf("Element not found");
}
getch();
}
1:1
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

Output:



Conclusion: Binary Search is a fast and efficient searching technique for sorted data. It requires fewer comparisons than Linear Search and is useful in real-world applications such as searching student records, employee IDs, and product codes.

Outcome: Successfully implemented the Binary Search technique to search for an element in a sorted dataset. Understood how Binary Search reduces searching time by dividing the array into halves.